AX,BX,CX,DX,SI,DI

AX=AH+AL=8+8=16

AH =255

AL=255

AX=65535=64k

EAX=32 Bits 4GB

RAX=64 Bits

inc ax

INC EAX

AX= AH +AL 16 bits AH=AL=8 Bits

16

EAX AX generico operaciones mate.

incrementar en 1

MOV EAX,1; EAX=1

INC EAX ;EAX=2

EBX,ECX,EDX,ESI, EDI

MOV EBX,0 ;EBX=0

INC EBX; EBX=1

SUMAR

ADD

MOV EAX,1; EAX=1

ADD EAX,3 ; EAX=4

ADD EAX,5 ; EAX=9

DECREMENTAR

DEC

MOV EAX,2; EAX=2

DEC EAX ; EAX =1

RESTAR

SUB

MOV EAX,2; EAX=2

SUB EAX,2 ; EAX=0

DIVIDIR

DIV

  MOV EAX,10 ;EAX=10

MOV AX, 10 ;AX=10

MOV BL,2; BH,BL

DIV BX ;EAX=10 / BX=2 =>5 RESIDUO =0

DX PARTE ALTA AX BAJA

7/2 -> 3 RESIDUO 1

MOV AL,10

MOV BL,2

DIV BL ; AL/BL =>10/2=>5 RESIDUO 0 AH=0

CMP AH,0

JE PAR ;JUMP EQUAL SI AX=0 SALTA A LA ETIQUETA PAR

```
;IMPRIMIR QUE ES IMPAR

; invoke MessageBox,0,addr about,addr titulo,0

Jmp salir

PAR:

;IMPRIMIR QUE ES PAR

; invoke MessageBox,0,addr about,addr titulo,0

Salir:



invoke ExitProcess,NULL



CMP AX,0

JE IMPAR          ;JUMP EQUAL

IMPRIMIR QUE ES PAR

Jmp salir

IMPAR:

IMPRIMIR QUE ES IMPAR

Salir:

invoke ExitProcess,NULL



MOV AX,7

CMP AX,0

JE CERO
```

```
MOV AX,10    ; AX=10

MOV BX,2     ;BX=2

DIV BX ; AX/BX =5
```

Using the .IF Syntax

MASM has the capacity to handle both simple and complex block comparison operations in a clear and easy to use syntax that reasonably well emulates higher level code. The .IF block syntax can be safely nested in the normal high level manner and is subject to the normal rules of comparison using the CMP instruction.

Simple

```
.if variable == value
  ; do something here
.endif
```

This type of code can be extended in two directions, it can handle more complex comparisons on a single line and it can perform multiple comparisons sequentially as is common in high level languages.

Sequential Evaluation

```
.if variable == value
  ; do something
.elseif variable != another_value
  ; do someting else
.else
  ; otherwise
.endif
```

You can repeatedly use the **.elseif** operator to perform additional sequential comparisons but you may only use one **.else** operator which must be the last option before the **.endif** operator.
Using the C comparison runtime operators below, more complex tasks can be performed

and in a notation that will be familiar to many programmers.

Complex Evaluation

```
.if variable > 50 && variable < 100
  ; number is within range
.else
  ; number is out of range
.endif
```

The code created by the .IF technique is competitive in quality terms to code created by the better end of compilers but note that it is a specific technique that may not be flexible enough for low level code design within an algorithm where register choice and instruction order become critical to the algorithm's performance.

C comparison run-time operators

Operator	Meaning
==	Equal
!=	Not equal
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to
&	Bit test (format: expression & bitnumber)
!	Logical NOT
&&	Logical AND
||	Logical OR
CARRY?	Carry bit set
OVERFLOW?	Overflow bit set
PARITY?	Parity bit set
SIGN?	Sign bit set
ZERO?	Zero bit set

The **.while** syntax is a clear code method for less than highly critical loop code where readability and ease of maintainance are the main considerations.

```
.while eax > 0
  sub eax, 1
.endw
```

The above **.while** loop produces the following code which shows the logic where the test is performed first by jumping to the end of the loop. This is the opposite of a

mov eax,2

```
 .repeat
  sub eax, 1
.until eax < 1
```

The above **.repeat** loop produces the following code which shows the logic where the test is performed after the code in the loop has been run This is the opposite of a **.WHILE** loop which is dealt with in the preceding topic.

```
lbl0:
  sub eax, 1
  cmp eax, 1
  jnb lbl0
```

```
include \masm32\include\masm32rt.inc

.data
Message1 db "2: Hello World Again!", 0
Message2 db "3: Hola Mundo!", 10,13,0
.code
start:
        cls

         print "1: Hello World!", 10, 13

         print addr Message1, 10, 13

         invoke StdOut, addr Message2

        ccout "\t C style string formatting\n\t in MASM"

        inkey

        invoke ExitProcess, 0

end start
```

SP-FFFF FFFF push EAX SP=SP-4

8	

Push EAX

Push CX

print “Hola Mundo”,10,13,0

EAX=11

Pop ECX

Pop EAX ->8

Sub EAX,1 -> 7

SP=0

include \masm32\include\masm32rt.inc

include \masm32\include\masm32rt.inc

```
.data
number DD 0
.code
start:
        cls
        mov EBX,[number] ;EBX=0
        .while EBX<=10

        mov [number],EBX    ;Number =1235
        print "The result is ="
        print str$(number),13,10,0
         inc EBX
         .endw
        inkey
        invoke ExitProcess, 0
end start
```

```
include \masm32\include\masm32rt.inc

.data

number DD 0

.code

start:

        cls

        mov EBX,[number] ;EBX=0

        .while EBX<10

        mov [number],EBX

        print "The result is ="

        print str$(number),13,10,0

          inc EBX

        .endw

        inkey

        invoke ExitProcess, 0




end start
```

```
RECT STRUCT
left DWORD ?
top DWORD ?
right DWORD ?
bottom DWORD ?
RECT ENDS
```

The notation for each member is membername, datasize, specifier. In most instances, the specifier is ? which means the member is not initialised to a value.

A structure is written in memory as a sequence of members so in the case of a RECT structure, it is written as four sequential DWORD size members in memory. The members of a structure can be filled in a number of different ways depending on how the structure was originally allocated.

If it is allocated in the .DATA section, it can be initialised to preset values but if it is allocated on the stack as a LOCAL value within a procedure, the values have to be coded into the structure.

```
LOCAL Rct :RECT

; code

mov Rct.left, 1
mov Rct.top, 2
mov Rct.right, 3
mov Rct.bottom, 4
```

It should be noted that a structure member is a memory operand so you cannot directly move another memory operand into it, you must either use a register to copy it or the stack mnemonics, push / pop.

You can now refer to the filled structure as a single unit with ADDR Rct in an invoke directive procedure call. If an API call requires the address of a structure, you fill the structure with the values you require and call the API,

```
invoke APIcall,parameter1,parameter2,ADDR Rct
```

If you write a procedure where you want the values in a structure passed to it, you can pass the structure by using the structures data type in the procedure.

```
MyProc proc par1:DWORD,par2:DWORD,MyRect:RECT

mov eax, MyRect.left ; copy first member into EAX
```

The individual members can then be accessed by their names in the procedure that receives the RECT as a parameter. You call the proc as follows,

```
invoke MyProc,par1, par2, Rct
```

Nested structures
A method common to 32 bit Windows is the use of nested structures. MASM has a notation for dealing with this type of construction. If you needed a structure that had multiple structures in it,

```
MyNestedStruct STRUCT
item1 RECT <>
item2 POINT <>
MyNestedStruct ENDS
```

This structure uses the above RECT struct and the following POINT struct.

```
POINT STRUCT
x DWORD ?
y DWORD ?
POINT ENDS
```

There are six members in this structure, four from the RECT structure and two from the POINT structure. Allocated on the stack it looks like this,

```
LOCAL mns:MyNestedStruct
```

The six members are,

```
mns.item1.left
mns.item1.top
mns.item1.right
mns.item1.bottom
mns.item2.x
mns.item2.y
```

The notation for mns.item2.x means the allocated structure mns , its second item item2 and the first item in the POINT struct x.

Structures can be nested to multiple depths but they all use this basic notation and logic.

Advanced Structure Handling

Increasingly, there is a need to be able to handle structures that are passed as an address and this type of code is becoming more common in Windows code design. MASM has a specialised notation to make this process a lot easier to handle.

If for example you needed to pass the adress of a RECT structure to a procedure, you would normally make the call in the following manner,

```
invoke MyFunction,ADDR Rct
```

At the procedure end of this function call you would normally have something like the following,

```
MyFunction proc lpRect:DWORD
```

With a simple structure like RECT you can address the seperate parameters manually by

putting the address into a register and write to the location of each member,

```
mov eax, lpRct

mov [eax], DWORD PTR 10
mov [eax+4], DWORD PTR 12
mov [eax+8], DWORD PTR 14
mov [eax+12], DWORD PTR 16
```

This works fine but with more complex structures this becomes much harder to work with and far more error prone.

The alternative is to use a method that MASM has to address the individual members by using the ASSUME directive.

```
ASSUME eax:PTR RECT
mov eax, lpRct
mov [eax].left, 10
mov [eax].top, 12
mov [eax].right, 14
mov [eax].bottom, 16
ASSUME eax:nothing
```

This works by telling the assembler that the EAX register is to be treated like a RECT structure. The ASSUME eax:nothing tells the assembler that the register is no longer being used in this manner.

There is an alternative notation where you can individually "type cast" each member.

```
mov eax, lpRct

mov (RECT PTR [eax]).left, 10
mov (RECT PTR [eax]).top, 12
mov (RECT PTR [eax]).right, 14
mov (RECT PTR [eax]).bottom, 16
```

FILE

fcreate

```
mov hFile, fcreate(filename)
```

Description
Create a new file with read / write access.

Parameters
1. **`filename`** A zero terminated string. This can be a quoted text.

Return Value
The return value is the handle of the new file.

Comments
The new file has read and write access. If there is an error, you should test the return value with **`GetLastError()`**.

Example
```
mov hFile, fcreate("mynewfile.ext")
```

fopen

```
mov hFile, fopen(filename)
```

Description
Open an existing file for read / write access.

Parameters
1. **`filename`** A zero terminated string. This can be a quoted text.

Return Value
The handle of the file.

Comments
If the function fails, you should test the return value with **`GetLastError()`**.

Example
```
mov hFile, fopen("existingfile.ext")
```

fwrite

```
mov bwritten, fwrite(hFile,buffer,bcnt)
```

Description
Write data to an open file.

Parameters
1. **`hFile`** The open file handle.
2. **`buffer`** The address of the buffer to write the data to.
3. **`bcnt`** The number of bytes to read.

Return Value
The number of bytes written to the file or zero on failure.

Comments
Use **`GetLastError()`** if the operation fails.

Example

```
mov bwritten, fwrite(hFile,lpmemory,bytecount)
```

fread

```
mov bWritten, fread(hFile,lpMemory,bytecount)
```

Description
Read data from an open file.

Parameters

1. **`hFile`** The open file handle.
2. **`buffer`** The address of the buffer to write the data to.
3. **`bcnt`** The number of bytes to read.

Return Value
The return value is the number of bytes read from the file.

Comments
The buffer must be large enough to handle the number of bytes read into it. Normally the buffer will be allocated to at least that size or larger.

Example
```
mov bWritten, fread(hFile,lpMemory,bytecount)
```

fseek

```
mov cloc, fseek(hFile,distance,location)
```

Description
Set the location of the file pointer.

Parameters
1. **`hFile`** The handle of the open file.
2. **`distance`** The number of bytes to move.
3. **`location`** The reference point to move from.

Return Value
The return value is the current location of the file pointer within the open file.

Comments
The **distance** argument can be specified as either a positive or a negative number which will be respectively added or subtracted from the position set in the **location** argument.

The **location** argument specifies what location in the file to use. There is a choice of 3 locations which are standard Windows equates.

`FILE_BEGIN` The beginning of the file.
`FILE_CURRENT` The current location of the file pointer.
`FILE_END` The end of the file

If for example you wished to set the file pointer at 100 bytes past the beginning of the file, you would use the equate **`FILE_BEGIN`** for the **`location`** argument and set the **`distance`** argument at 100.

Note that this macro will only work on files within the DWORD length. For files larger than 4 gigabytes use the **`SetFilePointer()`** API function.

Example

```
mov cloc, fseek(hFile,100,FILE_BEGIN)
```

fsize

```
mov flen, fsize(hFile)
```

Description
Returns the size in bytes of an open file.

Parameters
1. **`hFile`** The handle of the file.

Return Value
The length of the file in bytes.

Comments
The macro is limited to DWORD range. If you need to work on a file larger than 4 gigabytes, use the **`GetFileSize()`** API function.

fclose

```
fclose hFile
```

Description
Close an already open file handle.

Parameters
1. **`hFile`** The open file handle.

Return Value
Function has succeeded if the return value in EAX is not zero.

Comments
Errors should be tested with **`GetLastError().`**

Example
`fclose hFile`

```
.386
.model flat,stdcall
option casemap:none
include \masm32\include\windows.inc
include \masm32\include\user32.inc
includelib \masm32\lib\user32.lib
include \masm32\include\kernel32.inc
includelib \masm32\lib\kernel32.lib

.DATA
FileName db "C:\Users\machine\Desktop\putty.exe",NULL
BadText db "Its not ok",0
OkText db "Its ok",0
.DATA?
hFile HANDLE ?

.CODE
start:
	invoke CreateFile,addr FileName,GENERIC_READ OR GENERIC_WRITE,FILE_SHARE_READ OR
FILE_SHARE_WRITE, NULL,OPEN_EXISTING,FILE_ATTRIBUTE_NORMAL,NULL
	mov hFile,eax
	cmp hFile, INVALID_HANDLE_VALUE
	jz code1
	invoke MessageBox,NULL,addr OkText,addr OkText,MB_OK
	invoke ExitProcess,0



code1:
	invoke MessageBox,NULL,addr BadText,addr BadText,MB_OK
	invoke ExitProcess,0
	ret

end start
```

```
;
«««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««
«««««««««
    include \masm32\include\masm32rt.inc
;
«««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««
«««««««««

comment * -----------------------------------------------------
                      Build this  template with
                     "CONSOLE ASSEMBLE AND LINK"

        This example shows how to use the preprocessor code
        (macros) supplied with the latest service pack for
        MASM32 for file IO code.
        ------------------------------------------------------ *

    .code

start:

;
«««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««
«««««««««

    call main
    inkey
    exit

;
«««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««
«««««««««

main proc

    LOCAL hFile :DWORD                      ; file handle
    LOCAL bwrt  :DWORD                      ; variable for bytes
written
    LOCAL cloc  :DWORD                      ; current location
variable
    LOCAL txt   :DWORD                      ; text handle
    LOCAL flen  :DWORD                      ; file length variable
    LOCAL hMem  :DWORD                      ; allocated memory
handle

    sas txt,"Test String"                    ; assign string to
local variable

    .if rv(exist,"testfile.txt") != 0         ; if file already
exists
      test fdelete("testfile.txt"), eax       ; delete it
    .endif
```

```
  ; ---------------------------------
  ; create a file and write data to it
  ; ---------------------------------
    mov hFile, fcreate("testfile.txt")        ; create the file
    mov bwrt, fwrite(hFile,txt,len(txt))      ; write data to it
    fclose hFile                            ; close the file

  ; -------------------------------------
  ; reopen the file and append data to it
  ; -------------------------------------
    mov hFile, fopen("testfile.txt")          ; open the existing
file
    mov cloc, fseek(hFile,0,FILE_END)         ; set the file
pointer to the end
    fprint hFile," Additional data appended"   ; append text to
existing data
    fclose hFile                            ; close the file

  ; ------------------------------------------------
  ; open the file again, read and display its content
  ; ------------------------------------------------
    mov hFile, fopen("testfile.txt")          ; open the existing
file again
    mov flen, fsize(hFile)                   ; get its length
    mov hMem, alloc(flen)                    ; allocate a buffer of
that size
    mov bwrt, fread(hFile,hMem,flen)          ; read data from file
into buffer
    fclose hFile                            ; close the file

    invoke StripLF,hMem

    print hMem,13,10                         ; display text to
console

    free hMem                               ; free the allocated
memory

    ret

main endp

;
«««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««
«««««««««

end start



.386
```

```
.model flat,stdcall
option casemap:none

include    \masm32\include\windows.inc
include    \masm32\include\kernel32.inc
include    \masm32\include\user32.inc
include    \masm32\include\masm32.inc

includelib  \masm32\lib\kernel32.lib
includelib  \masm32\lib\user32.lib
includelib  \masm32\lib\masm32.lib


.data

FileName    db 'test.txt',0    ; file to read

.data?

hFile      dd ?
FileSize    dd ?
hMem       dd ?
BytesRead   dd ?

.code

start:


   invoke  CreateFile,ADDR FileName,GENERIC_READ,0,0,\
          OPEN_EXISTING,FILE_ATTRIBUTE_NORMAL,0

   mov    hFile,eax

   invoke  GetFileSize,eax,0

   mov    FileSize,eax
   inc    eax

   invoke  GlobalAlloc,GMEM_FIXED,eax
   mov    hMem,eax

   add    eax,FileSize

   mov    BYTE PTR [eax],0   ; Set the last byte to NULL so that
StdOut
                             ; can safely display the text in memory.

   invoke  ReadFile,hFile,hMem,FileSize,ADDR BytesRead,0

   invoke  CloseHandle,hFile
```

```
   invoke  StdOut,hMem

   invoke  GlobalFree,hMem

   invoke  ExitProcess,0

END start
---------------------------------
Otro Ejemplo es otro program
---------------------------------
.386
.model flat, stdcall
option casemap :none

Include   windows.inc
Include   user32.inc
Include   kernel32.inc
IncludeLib   user32.lib
IncludeLib   kernel32.lib

.data
theFile DB "n_Copy.exe", 0
inputText DD 0
inputFile DB "C:\WINDOWS\NOTEPAD.EXE", 0

iFile DD 0
File DD 0
written DD 0
iwritten DD 0
fSize DD 0

.code
start:



;=================== open file ===============;
       push 0
       push FILE_ATTRIBUTE_NORMAL
       Push OPEN_EXISTING
       push 0
       push FILE_SHARE_READ
       Push GENERIC_READ
       Push Offset inputFile
       call CreateFile
       Mov iFile, Eax
;=================== set write head at 0 ===============;
       Push FILE_BEGIN
       push 0
       push 0
       Push iFile
```

```
       call SetFilePointer

       Push 0
         Push iFile
         Call GetFileSize
       ; at this point eax should be equal to the file size right?

;=================== read from file ===============;
        Push 0
        Push Offset iwritten
        Push Eax ;
        Push Offset inputText
        Push iFile
        Call ReadFile
;=================== close file ===============;
       Push iFile
       Call CloseHandle


;=================== create file ===============;
       Push 0
       push FILE_ATTRIBUTE_NORMAL
       push OPEN_ALWAYS
       push 0
       push FILE_SHARE_READ
       push GENERIC_WRITE
       push offset theFile
       call CreateFile
       mov File,eax
;=================== set write head at 0 ===============;
       push FILE_END
       push 0
       push 0
       push File
       call SetFilePointer
;=================== get string length ===============;
       Push Offset inputText
       call lstrlen
;=================== write to file ===============;
        push 0
        Push Offset written
        Push iwritten       ;iwritten how holds the number of
bytes read...
        Push Offset inputText
        push File
        call WriteFile
;=================== close file ===============;
       push File
       call CloseHandle
;=================== exit app ===============;
       invoke ExitProcess,eax
       invoke PostQuitMessage,0
```

```
end start
```

Ejemplo escribir en el archivo

```
       INCLUDE \masm32\include\masm32rt.inc

       .DATA

szFileName db 'MyFile.txt',0

       .DATA?

hFile  HANDLE ?
nBytes dw ?

       .CODE

_main   PROC

       mov    eax,12345678h
       mov    edx,uhex$(eax)    ;EDX = address of hexidecimal
string

;create the file

       push   edx
       INVOKE  CreateFile,offset
szFileName,GENERIC_WRITE,FILE_SHARE_READ,
              NULL,CREATE_ALWAYS,FILE_ATTRIBUTE_NORMAL,NULL
       mov    hFile,eax
       pop    edx

;write the file

       INVOKE  WriteFile,eax,edx,8,offset nBytes,NULL

;close the file

       INVOKE  CloseHandle,hFile

;terminate

       INVOKE  ExitProcess,0

_main   ENDP

       END    _main
```